Ε.Μ.Π. - ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΑΚΑΔ. ΕΤΟΣ 2024-2025

ΑΘΗΝΑ 15 - 05 - 2026

3η ΟΜΑΔΑ ΑΣΚΗΣΕΩΝ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Συστήματα Μικροϋπολογιστών"

Παράδοση 31/05/2026
Ασκήσεις προσομοίωσης της αρχιτεκτονικής ARM

Όπου ζητείται να γίνει χρήση του **STM32CubeIDE** με το πρόγραμμα προσομοίωσης **Renode** και το **GUI tool**

1η ΑΣΚΗΣΗ: Επιθυμούμε να κατασκευάσουμε ένα ARM Cortex-M4 SoC (STM32F407VGT6) ενσωματωμένο σε ενσωματωμένη συσκευή αλληλούχισης RNA, το οποίο θα επιτελεί τις εξής λειτουργίες:

- Έχει αποθηκευμένη την αλληλουχία RNA του ιού SARS-CoV-2 (COVID-19). Η αλληλουχία αυτή μπορεί να βρεθεί εδώ στο πεδίο **ORIGIN**.
- Ο χρήστης να δίνει μέσω του περιφερειακού **USART2**, μια συμβολοσειρά η οποία θα αποτελεί μία αλληλουχία νουκλεοτιδίων RNA που θα ήθελε να ψάξει το συμπλήρωμά της στην αποθηκευμένη αλληλουχία του ιού του COVID. Το τέλος της εισαγωγής input από τον χρήστη **θα δίνεται με την αλλαγή γραμμής ('\n')**.
- Αν σε οποιαδήποτε στιγμή κατά την εισαγωγή συμβολοσειράς **πατηθεί το πλήκτρο GPIOD4 (PD4)**, θα πρέπει να συμβαίνει **interrupt στην θέση EXTI0** του επεξεργαστή, η οποία **θα ανάβει το LED στο GPIOA13 (PA13) για 3 δευτερόλεπτα και θα ξεκινάει την αναζήτηση από την αρχή**.
- Μετά την εισαγωγή της συμβολοσειράς από τον χρήστη, το πρόγραμμα θα στέλνει μέσω USART2, **πόσες φορές βρέθηκε** η αλληλουχία καθώς και **τις θέσεις στις οποίες βρέθηκε** (θέση του πρώτη νουκλεοτιδίου στο κάθε αποτέλεσμα).
- Άμα βρεθεί τουλάχιστον μία αλληλουχία (πετύχει η αναζήτηση) θα ανάβει **το LED στο GPIOA12 (PA12) για 2 δευτερόλεπτα.**
- Το πρόγραμμα **θα επαναλαμβάνει ανεάως** αυτή τη διαδικασία.

Ζητείται να υλοποιηθεί η παραπάνω διαδικασία **σε C και σε Assembly**. Στην υλοποίησή σας **σε assembly**, το πάτημα του πλήκτρου **GPIOD4 (PD4) θα διαβάζεται κανονικά από το πρόγραμμα (polling) αντί να προκαλεί interrupt** για περεταίρω διευκόλυνση, επιτελώντας όμως την ίδια λειτουργία (θα ανάβει το LED στο GPIOA13-PA13 για 3 δευτερόλεπτα και θα ξεκινάει την αναζήτηση από την αρχή).

Για την εκτέλεση του προγράμματος θεωρήστε ότι η κάθε αλληλουχία θα εκφράζεται σαν μία σειρά από **chars** με πιθανές τιμές **'A' (Αδενίνη), 'G' (Γουανίνη), 'U' (Ουρακίλη), 'C' (Κυτοσίνη)**. Θυμίζουμε ότι **τα συμπληρωματικά ζευγάρια στο RNA είναι A-U και G-C**. Το πρόγραμμά σας πρέπει να είναι **case-sensitive** και να δέχεται σαν έγκυρους **μόνο τους χαρακτήρες 'A', 'G', 'U', 'C', '\n'**. Οποιοσδήποτε άλλος χαρακτήρας μπαίνει στην είσοδο **θα πρέπει να αγνοείται**.

Γενική Παρατήρηση 1: *Τα προγράμματα να συνοδεύονται υποχρεωτικά στα κυριότερα σημεία τους από σύντομα σχόλια.*
Γενική Παρατήρηση 2: *Για επαλήθευση της ορθής λειτουργίας των προγραμμάτων, να συμπεριλάβετε ευκρινή screenshots επιλεγμένων σεναρίων, τα οποία πρέπει να συνοδεύονται υποχρεωτικά με την αντίστοιχη επεξήγηση του σεναρίου.*

Θεωρητικές Ασκήσεις

2η ΑΣΚΗΣΗ: Σε ένα σύστημα σε ψηφίδα (SoC) βασισμένο στον επεξεργαστή ARM-Cortex M4, θέλουμε να κατασκευάσουμε ένα πρόγραμμα με τα εξής χαρακτηριστικά:

- Το section **.text θα είναι πάντα = 1KB**
- Το section .rodata θα είναι πάντα <= 512MB
- Το section .data θα είναι πάντα <= 512MB
- Υπάρχει επικοινωνία με external RAM, στην οποία μεταφέρονται πάντα πακέτα συνολικής χωρητικότητας <= 128MB
- Διαθέτουμε 4 GPIO Devices (περιφερειακά) από τα οποία το κάθε ένα διαθέτει 5 configuration registers των 32-bit

Καλείστε να απαντήσετε στις παρακάτω σχεδιαστικές ερωτήσεις:
(α) Ποια είναι η μικρότερη χωρητικότητα των Flash Memory και SRAM που μπορεί να υποστηρίξει το SoC μας χωρίς να υπάρχει σφάλμα στην εκτέλεση;
(β) Σε ποια address spaces θα χαρτογραφήσουμε τα .text, .rodata, .data regions, καθώς και τα GPIO περιφερειακά μας και την external RAM; Αιτιολογήστε την απάντησή σας βασιζόμενοι **στο datasheet του ARM Cortex-M4**.

3η ΑΣΚΗΣΗ: Σε ένα σύστημα σε ψηφίδα έχουμε τοποθετήσει ένα περιφερειακό το οποίο διαχειρίζεται μία φωτοδίοδο (LED). Το περιφερειακό επικοινωνεί με τον host ARM επεξεργαστή μέσω γέφυρας διαδρόμου **AMBA-APB**. Για την σωστή λειτουργία του περιφερειακού έχουμε υλοποιήσει 2 καταχωρητές ελέγχου:

- **LED_CTRL**, Εγγραφή νέας τιμής στο LED: Mapped in address **0x40000000**
  - 1 κύκλος για ολοκλήρωση
- **LED_STATUS**, Ανάγνωση της υπάρχουσας τιμής στο LED: Mapped in address **0x40000004**
  - 2 κύκλοι για ολοκλήρωση

Δίνεται ο παρακάτω κώδικας ARM assembly:

```
        .syntax unified
        .thumb
        .global main

        .equ LED_CTRL,   0x40000000
        .equ LED_STATUS, 0x40000004

main:
        LDR     r0, =LED_CTRL
        MOVS    r1, #1
        STR     r1, [r0]

        LDR     r0, =LED_STATUS
        LDR     r2, [r0]

stop:
        B       stop
```

(α) Για κάθε μεταφορά δεδομένων από και προς το περιφερειακό (μέσω του APB Bus), να σχεδιαστεί αντίστοιχο διάγραμμα χρονισμού (timing diagram). Προαιρετικά προτείνεται να χρησιμοποιηθεί η πλατφόρμα WaveDrom Editor.

(β) Να σχεδιαστεί κατάλληλο FSM το όποιο θα υλοποιεί τον ελεγκτή διεπαφής του APB περιφερειακού, δηλ. Το FSM που θα ελέγχει την κατάσταση των εισόδων του APB περιφερειακού, και θα παράγει με σωστό χρονισμό τα σήματα: PREADY, PRDATA βάση της προδιαγραφής του χρονισμού του APB.